A MINI PROJECT REPORT

ON

SMART CROP
DASHBOARD USING PYTHON

Crop Suitability Screening & Interactive Data Visualization

Submitted in partial fulfillment of the requirements for

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING (AI / AIML)

SUBMITTED BY

G.H.NAVEEN	24FH1A3332
M.KARTHIK	24FH1A3350
p.shashavalli	24FH1A3355
k.vasanth	24FH1A3344
P.Harsha Vardhan	24FH1A3354

Under the esteemed guidance of

DR. B. MAHESH

Associate Professor, Department of CSE

DEPARTMENT OF CSE (AI & AIML)

DR. K. V. SUBBA REDDY INSTITUTE OF TECHNOLOGY

KURNOOL | UGC AUTONOMOUS

Approved by AICTE, New Delhi | Affiliated to JNTUA, Anantapur

ACADEMIC YEAR 2026-2027

DR. K. V. SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS | DEPARTMENT OF CSE (AI / AIML)

CERTIFICATE

This is to certify that the mini project report titled "SMART CROP DASHBOARD USING PYTHON" is a project document prepared by the following students:

G.H.NAVEEN	24FH1A3332
M.KARTHIK	24FH1A3350
p.shashavalli	24FH1A3355
k.vasanth	24FH1A3344
P.Harsha Vardhan	24FH1A3354

The report is submitted to the Department of Computer Science and Engineering (AI / AIML), Dr. K. V. Subba Reddy Institute of Technology, Kurnool, in partial fulfillment of the requirements for the Bachelor of Technology mini-project academic activity.

Guide: Dr. B. Mahesh, Associate Professor, Department of CSE. The project source and demonstrations are described in the accompanying technical chapters.

Academic submission template: guide and departmental signatures should be added after review and approval. No signatures are reproduced here.

Project Guide

Head of Department

DR. K. V. SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS | DEPARTMENT OF CSE (AI / AIML)

DECLARATION

We declare that the mini project report entitled "SMART CROP DASHBOARD USING PYTHON" presents the application design, scoring method, visualizations and conclusions based on the accompanying Smart_Crop_Dashbord.py file and a review of its embedded web interface.

The numerical outputs reported for the three example fields are generated from the code-provided demonstration conditions. They are not field-trial observations, machine-learning predictions, verified crop yields or guaranteed agricultural recommendations.

Where external agricultural profiles are referenced by the application, their use is described as an implementation detail rather than independent verification of agronomic accuracy. Any final academic declaration should be signed by the students after review.

G.H.NAVEEN	24FH1A3332
M.KARTHIK	24FH1A3350
p.shashavalli	24FH1A3355
k.vasanth	24FH1A3344
P.Harsha Vardhan	24FH1A3354

ACKNOWLEDGEMENT

We express our sincere gratitude to the management and faculty of Dr. K. V. Subba Reddy Institute of Technology, Kurnool, for providing an academic environment that encourages practical learning and project development.

We are thankful to Dr. B. Mahesh, Associate Professor, Department of CSE, for the guidance and support associated with the mini-project report format supplied as our reference.

We also acknowledge our department teachers, classmates and family members for their encouragement throughout the preparation, analysis and documentation of the Smart Crop Dashboard project.

This report explains an educational crop suitability tool. We recognize the importance of interpreting model results carefully and confirming any agricultural decision with reliable local expertise.

G.H.NAVEEN	24FH1A3332
M.KARTHIK	24FH1A3350
p.shashavalli	24FH1A3355
k.vasanth	24FH1A3344
P.Harsha Vardhan	24FH1A3354

REPORT GUIDE / TABLE OF CONTENTS

This technical report follows the same 14-section academic structure as the supplied KVSR Smart Campus example, but all software descriptions and computed sample outputs are drawn from the submitted **Smart_Crop_Dashbord.py**. The cover, certificate, declaration and acknowledgement follow the institutional style of the reference document.

Application verified from the uploaded file

The supplied project is a **single-file Python local web server** that serves an embedded HTML/CSS/JavaScript dashboard. It is **not a Streamlit, Pandas, Plotly or trained machine-learning application**. Python 3.9+ is required only for serving the bundled page; browser JavaScript performs the scoring.

01	Abstract
02	Introduction
03	Literature Survey / Existing System
04	Technologies and Tools Used
05	System Requirements
06	Dataset Description
07	Implementation
08	Data Visualization
09	Results and Discussion
10	Advantages and Limitations
11	Conclusion
12	Future Enhancement
13	References
14	Appendix (source, tests and screenshots)

How to read the evidence

- Code-confirmed features: six menu sections, eight crop profiles, five weighted factors, three demo field presets, scenario lab, searchable crop library, batch CSV and exports.
- Charts in chapters 8 and 9 reproduce rule-engine outputs for supplied demonstration inputs; they are not field results or independent validation.
- Screenshots in Appendix C were captured from the embedded dashboard supplied in the Python file.

01 Abstract

The **Smart Crop Dashboard** is an offline-capable crop suitability screening application developed as a single Python file containing a complete responsive web interface. It allows a user to describe one agricultural field using average growing-season temperature, annual rainfall, soil pH, soil texture and drainage. A transparent, rule-based scoring method evaluates eight crop profiles and presents a ranked suitability index, limiting factors, interactive comparisons, hypothetical scenarios and downloadable results.

The dashboard uses a lightweight Python **ThreadingHTTPServer** to serve its embedded page on the local computer. HTML, CSS and client-side JavaScript provide charts, forms, tables and CSV processing. No external Python packages, API keys, machine-learning model, live database or internet connection are required for its core calculations.

Under the built-in dryland demonstration (29°C; 550 mm/year; pH 6.8; medium texture; well drained), Sorghum ranks first at **100.00** and six of eight crops fall into the strong-match band. The sample rankings are illustrative screening scores, not crop yield or profit forecasts.

Keywords: smart agriculture; crop suitability; rule-based decision support; JavaScript visualization; Python HTTP server; scenario analysis; CSV screening.

02 Introduction

2.1 Background and motivation

Farmers and students may need a compact way to compare crops against environmental measurements. Manual crop comparisons can require consulting multiple reference tables and interpreting temperature, rainfall and soil properties independently. This prototype consolidates a small collection of reference-profile ranges and uses a consistent scoring procedure to explain their match with a field.

2.2 Problem statement

Design a transparent, browser-based agricultural dashboard that accepts basic soil and climate inputs, evaluates pre-defined crop profiles, visualizes the resulting suitability scores, and supports simple what-if analysis and CSV reporting without requiring a paid service or complex deployment.

2.3 Project objectives

- Provide a validated form for field inputs and three ready-to-load demonstration scenarios.
- Compute explainable crop rankings across eight profiles and five weighted factors.
- Show blocked factors and match bands, not unexplained black-box recommendations.
- Provide overview, crop comparison, scenario analysis, crop library, batch screening and export views.
- Run locally on a laptop with Python 3.9+ and no third-party Python dependencies.

2.4 Scope

The deliverable is intended for educational, exploratory screening. It does not predict yield, selling price, crop duration, profitability, irrigation requirements or the best cultivar for an individual village. It does not use live weather feeds, official soil tests or geolocation.

03 Literature Survey / Existing System

3.1 Existing approaches

Approach	Typical value	Project-specific trade-off
Manual crop reference tables	Agronomic guidance can be read independently	Repeated cross-checking of five variables
Spreadsheet decision matrix	Customizable scoring and charts	User must maintain formulas and filters
Remote agriculture advisory portal	May integrate local information	May rely on internet, credentials or paid services
Smart Crop Dashboard	Standalone interface with visible factors and rules	Eight profiles only; reference assumptions are simplified

3.2 Gap identified

For a classroom demonstration, there is value in a lightweight tool that behaves consistently without a network connection and visibly identifies how each factor affects a ranking. The present system prioritizes traceable scoring and exploration rather than claims of field-tested agronomic accuracy.

3.3 Proposed system

The proposed workflow collects a field description; validates values; compares each field measurement against a crop profile; applies five numerical/categorical component scores; blocks a crop on an unsupported factor; and presents a classified, sorted crop list with reports and what-if comparisons.

04 Technologies and Tools Used

Technology	Function	Implementation
Python 3.9+	Starts a local web server	Standard library
http.server	Serves the dashboard and /health endpoint	ThreadingHTTPServer
HTML5 / CSS3	Semantic sections, forms and responsive interface	Embedded in Python payload
JavaScript	Scoring, filtering, SVG and DOM updates	Runs in browser
SVG / CSS charts	Ranks, donut, sensitivity plot	Native front-end rendering
Browser APIs	Client-side CSV load and file downloads	File/Blob APIs
zlib + base64	Stores compressed HTML document in the Python file	Python standard library
Streamlit / Pandas / Plotly	Not required by this uploaded version	Not imported or used

Deployment distinction

The report names and installation instructions match **Smart_Crop_Dashbord.py** as uploaded. The docstring includes an older example filename, Smart_Crop_Web_App.py, but users should execute the **actual saved file name**.

05 System Requirements

5.1 Hardware and software

Component	Recommended configuration
Computer	Windows, macOS or Linux laptop/desktop; modern dual-core CPU
Memory	Approximately 2 GB or more; ordinary browser usage
Python	Python 3.9 or newer
Browser	Recent Chrome, Edge, Firefox or Safari with JavaScript enabled
Internet	Not needed for core calculations; external FAO profile links need access
Python packages	No pip installs required; uses only Python built-in modules

5.2 Installation and running on Windows

Step 1. Install Python 3 from python.org. Tick “Add Python to PATH” during setup when offered.

Step 2. Place Smart_Crop_Dashbord.py in a folder such as Downloads.

Step 3. Open Command Prompt in that folder.

Step 4. Run: python Smart_Crop_Dashbord.py

Step 5. Open http://127.0.0.1:8501 if the browser does not open automatically.

Step 6. Keep the terminal open; press Ctrl+C to stop the app.

Windows command

Command	Purpose
`python --version`	Check Python installation
`python Smart_Crop_Dashbord.py`	Launch the local dashboard
`python Smart_Crop_Dashbord.py --port 8600`	Choose a different permitted port

5.3 macOS/Linux

Open a terminal in the project folder and use `python3 Smart_Crop_Dashbord.py`. The dashboard binds to 127.0.0.1, which is visible only on the same computer.

06 Dataset Description

6.1 Actual data source

The uploaded application contains an **embedded static profile dictionary** (DATA.PROFILES), not an external spreadsheet, machine-learning training dataset or live agricultural database. It defines exactly eight crop records with numeric range thresholds, acceptable soil categories and scientific names. It additionally contains three synthetic field presets for repeatable demonstration.

Input field	Accepted values / validation	Meaning
field_name	Nonempty text, max 100 characters	Name of the submitted field
temp_c	−20 to 60 °C	Growing-season mean temperature
rain_year_mm	0 to 12,000 mm	Annual rainfall total
ph	0 to 14	Soil pH
texture	Light / Medium / Heavy / Organic	Categorical soil texture
drainage	Well / Excessively / Poorly drained	Categorical drainage label
origin	Demonstration or user-supplied	Input provenance label

6.2 Crop profile coverage

Crop	Scientific name	Group
Rice	Oryza sativa	Cereals
Maize	Zea mays ssp. mays	Cereals
Wheat	Triticum aestivum	Cereals
Sorghum	Sorghum bicolor	Cereals
Pearl millet	Pennisetum glaucum	Cereals
Groundnut	Arachis hypogaea	Oilseeds
Sunflower	Helianthus annuus	Oilseeds
Chickpea	Cicer arietinum	Pulses

Embedded demonstration field presets

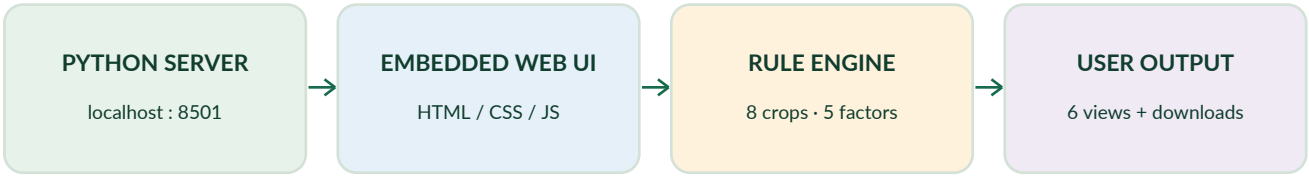
Preset	Temp	Rain	pH	Texture / drainage
Dryland	29°C	550 mm	6.8	Medium / well drained
Wetland	27°C	1,700 mm	6.2	Heavy / poorly drained
Cool climate	20°C	800 mm	6.4	Medium / well drained

The code labels its crop profile source snapshot as **8 October 2026** and links to FAO EcoCrop data sheets by crop ID. These are references embedded in the program; this project report does not independently verify the profile ranges against current FAO guidance.

07 Implementation

7.1 Single-file application architecture

The Python entry point decodes compressed page bytes with **base64** and **zlib**, then serves that web page using a standard-library HTTP server. All calculations run in the user's browser. There is no remote server, cloud connection or server-side scoring endpoint.



The browser executes scoring locally; files are downloaded through browser APIs.

Figure 1. Actual project architecture reconstructed from the supplied Python and embedded JavaScript.

7.2 Web server and local access

- GET / and GET /index.html return the bundled dashboard; GET /health returns a small JSON status response.
- The server binds to 127.0.0.1, not the public network; startup uses port 8501 by default and tries nearby ports if busy.
- Optional --no-browser avoids launching a browser automatically; --port selects a starting port between 1024 and 65535.

7.3 Suitability scoring algorithm

For each crop, the dashboard evaluates three numerical factors (temperature, annual rainfall and pH) plus two categorical factors (soil texture and drainage). A numerical factor scores 100 inside its preferred interval, tapers linearly toward 20 at the allowable endpoints, and becomes zero outside the allowable limits. A categorical input scores 100 if preferred, 60 if tolerated and zero if unsupported.

Weighted score and blocking rule

Index = 0.30 × temperature + 0.30 × rainfall + 0.25 × soil pH + 0.10 × texture + 0.05 × drainage. If **any** one component scores zero, the overall index is forced to **0**. The index is a heuristic ranking score, not a confidence or success probability.

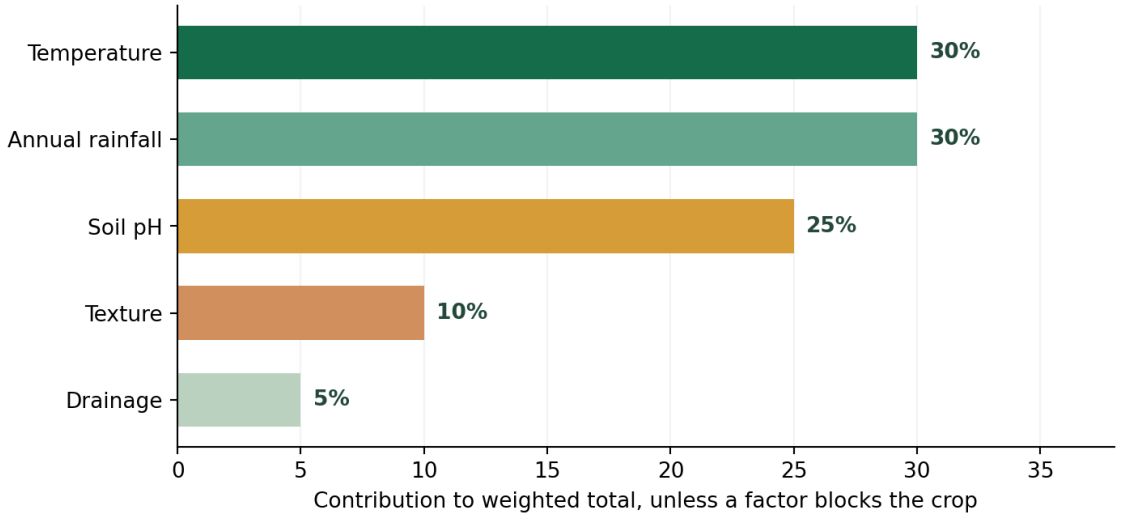


Figure 2. Weights read directly from the rule engine (sum = 100%).

7.4 Match classification

Band	Threshold	User-facing meaning
Strong match	Index ≥ 80	Fits the configured profile relatively well
Moderate match	60 ≤ index < 80	Partial compatibility under reference assumptions
Limited match	0 < index < 60	Multiple or sizable deviations
Outside profile	Index = 0	One or more hard blocking conditions

7.5 Dashboard module navigation

Menu	Core functionality
Overview	KPIs, three-card shortlist, ranking bars, outcome donut and factor table
Compare	Up to four selected crops; five-factor table and individual detail view
Scenarios	Temperature, rainfall and pH sliders; baseline/scenario comparison and sensitivity line
Crop library	Search and filter all eight crop profiles by group
Batch CSV	Template, upload, validate, rank up to 2,000 fields, export results
Reports	Download results CSV, printable HTML, and session-only snapshots

08 Data Visualization

The included application renders interactive ranking bars, a CSS-based outcome donut, factor score tables and an SVG temperature-sensitivity line. The figures below reproduce those analytical values for print documentation and show static equivalents, not live charts or independent observations.

8.1 Bar chart - dryland crop rankings

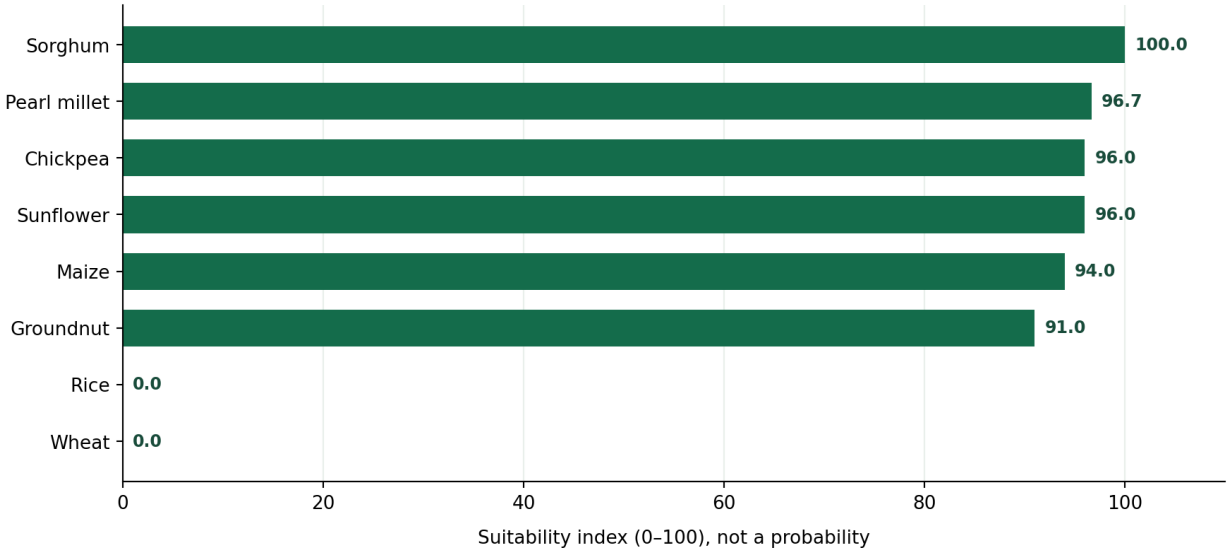


Figure 3. Dryland demonstration profile: sorghum leads at 100.00; blocked crops are assigned zero.

The built-in dryland field is 29°C, 550 mm/year, pH 6.8, with medium-textured well-drained soil. Six crops receive a strong match. Rice is blocked by rainfall and drainage; Wheat is blocked by temperature under the preset rule ranges.

8.2 Donut chart - match outcome distribution



Figure 4. Dryland classification: six strong matches, zero moderate, zero limited and two outside profile.

8.3 Heatmap - comparisons across three built-in field presets



Figure 5. Rule-engine suitability indices for dryland, wetland and cool-climate examples; zeros mean at least one factor blocks the crop.

The wetland example produces Rice at 100 and every other tested crop at zero because of its configured rainfall, texture and/or drainage restrictions. The cool-climate example gives equal top scores (100) to Chickpea, Maize, Sunflower and Wheat. Ties must be understood as equivalent screening indices, not evidence of equal real yields.

8.4 Sensitivity curve - Sorghum against hypothetical temperature

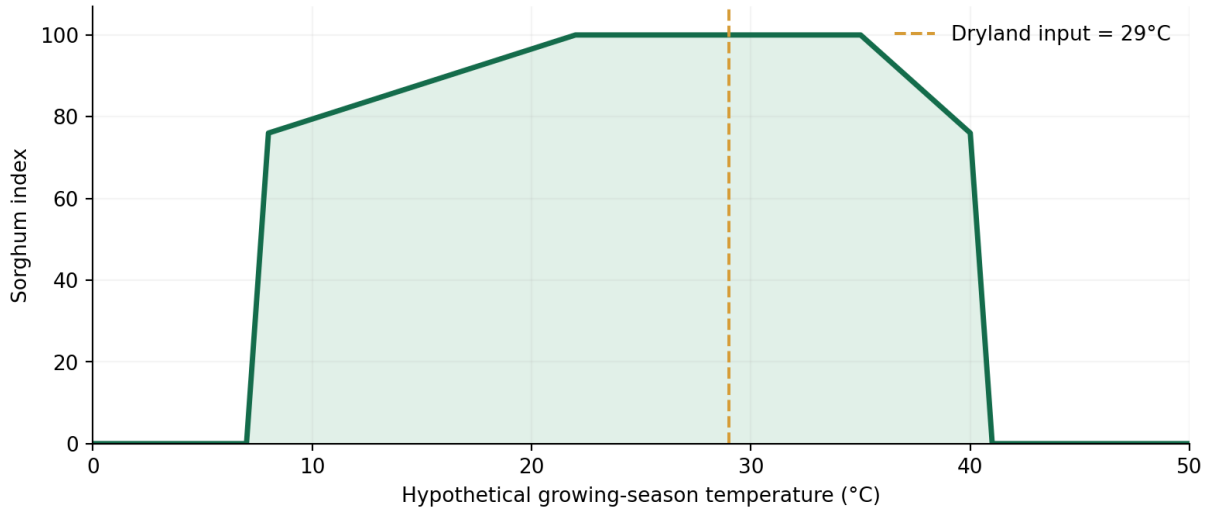


Figure 6. One-variable sweep holds the other four dryland field inputs constant and varies temperature.

The scenario laboratory additionally offers changes in temperature (−10 to +10°C), annual rainfall (−80% to +100%) and pH (−2 to +2). This is a what-if exploration, not a weather or climate prediction.

09 Results and Discussion

9.1 Test setup

The observations in this chapter come from executing the uploaded JavaScript evaluator on its three embedded field presets. They are deterministic outputs for fixed code and inputs; no real-world crop production measurements were collected or validated.

9.2 Results by demonstration field

Field preset	Top result(s)	Score	Strong / Outside
Dryland	Sorghum	100.00	6 / 2
Wetland	Rice	100.00	1 / 7
Cool climate	Chickpea, Maize, Sunflower, Wheat (tie)	100.00 each	6 / 2

9.3 Detailed dryland screening

Rank	Crop	Suitability index	Classification
1	Sorghum	100.00	Strong match
2	Pearl millet	96.67	Strong match
3	Chickpea	96.00	Strong match
4	Sunflower	96.00	Strong match
5	Maize	94.00	Strong match
6	Groundnut	91.00	Strong match
7	Rice	0.00	Outside profile
8	Wheat	0.00	Outside profile

9.4 Interpretation

Sorghum receives 100 because each submitted dryland factor lies in the program's preferred band or category. Pearl millet receives 96.67 because its soil pH factor is below the maximum component score, while the remaining dimensions are favorable. These explanations are calculable from the program, rather than inferred from a trained model.

A zero index should be interpreted as "outside one or more coded allowable bands", not as proof that cultivation is impossible. Conversely, a 100 index is not a guarantee of germination, yield or profitability. Crop variety, water timing, nutrient supply, pests and management conditions can fundamentally change actual outcomes.

9.5 Usability and validation checks

Test	Expected behavior	Observed in source
Empty or invalid numeric input	Display validation feedback	Explicit range checks implemented
Unsupported soil category	Reject invalid entry	Membership validation implemented
CSV > 2 MB	Reject file	Size guard implemented
Batch > 2,000 rows	Reject or limit upload	Row-count validation implemented
Tied highest score	Show all tied crops	Tie aggregation implemented
Any factor zero	Overall crop index zero	Hard blocking rule implemented
Saved snapshots	Keep in page session only	No durable persistence implemented

10 Advantages and Limitations

10.1 Advantages

- Runs on a local laptop with no third-party Python dependencies or account setup.
- Transparent component scoring, limiting-factor descriptions and well-defined classification bands.
- Multiple dashboard views including comparisons, what-if exploration, a crop library and printable reports.
- Bulk CSV screening avoids manual one-by-one evaluation for classroom-sized sample collections.
- Offline core functionality with source reference links available when connected.

10.2 Limitations

- Only eight predefined crop profiles are present in the uploaded program.
- No trained model, official prediction accuracy, experimental validation or locality-specific agronomic calibration.
- Rainfall is annual climate rainfall, not effective precipitation or irrigation demand.
- NPK, salinity, crop variety, extreme weather, pest pressure, crop calendar, yield and economics are not modelled.
- No database, authentication, cloud sync or permanent saved-field history; snapshots reset after reload.
- Localhost service cannot automatically be opened from another phone or remote device.

11 Conclusion

The Smart Crop Dashboard demonstrates how a single Python file can package a responsive, offline-capable agricultural decision-support interface. Its combination of input validation, explainable crop-profile scoring, comparisons, what-if scenarios, file-based batch processing and exports makes it suitable for project demonstrations and learning how to design transparent data-visualization workflows.

The greatest strength is the clarity of its rules and portability. Its most important limitation is that the index is a simplified educational screening measure, not a verified agronomic recommendation. Practical crop decisions require locally validated expert advice and field-specific information.

12 Future Enhancement

Proposed improvement	Expected benefit / prerequisite
Expand crop library	Include additional vegetables, pulses, fruits and regional crops after agronomist review
Soil laboratory integration	Use verified NPK, EC, organic matter and soil-test data
Localized rainfall and forecast feeds	Include timing, seasonal variability and water availability; requires trusted APIs
Location and crop calendar	Account for region, sowing season and cultivar selection
Yield/cost analysis	Separate evidence-backed yield estimates and crop economics from suitability index
Database and user accounts	Save fields safely with authentication, privacy controls and data governance
Validation study	Compare recommendations against agronomy experts and independent field observations
Accessibility/local languages	Add language options, high-contrast modes and printable farmer guides

13 References

Primary project evidence

1. **Smart_Crop_Dashbord.py**, uploaded single-file Python project (embedded HTML, CSS, JavaScript, crop profiles and scoring rules), examined for this report.

2. **KVSR_Smart_Campus_.pdf**, uploaded sample academic report, used as the structural and institutional-layout reference.

Program-described external reference links

3. FAO EcoCrop web database. The application provides individual crop data-sheet links at https://ecocrop.apps.fao.org/ecocrop/. The application labels the copied reference-profile snapshot “8 October 2026”.

Platform documentation for further reading

4. Python Standard Library - http.server, base64 and zlib: https://docs.python.org/3/library/http.server.html.

5. MDN Web Docs - HTML forms, JavaScript File API, Blob and SVG: https://developer.mozilla.org/.

14 Appendix

Appendix A Source-code structure

Code block	Responsibility
Imports and _DATA payload	Python standard-library imports and compressed embedded web page
DashboardHandler	Returns HTML, health JSON and HTTP error responses
DashboardServer / main	Binds loopback address, opens browser, manages lifecycle
DATA constants	Eight crop profiles, factors, weights and presets
validate / rangeScore / evaluate	Field input checks and deterministic crop rule engine
overview / compare / scenarios	Dashboard views and in-browser interactive visuals
library / batchEvaluate	Reference cards, CSV parsing and bulk ranking
reports / snapshots	CSV, printable HTML and temporary browser-session captures

Appendix B Installation, CSV schema and basic verification

Run the provided file from its actual directory, then browse to **http://127.0.0.1:8501**. The Python console prints the address if the default port is occupied. The browser uses JavaScript, so it must be enabled.

Platform	Launch command
Windows	`python Smart_Crop_Dashbord.py`
macOS / Linux	`python3 Smart_Crop_Dashbord.py`
Custom port	`python Smart_Crop_Dashbord.py --port 8600`

Required batch CSV columns

```
field_name, temp_c, rain_year_mm, ph, texture, drainage
```

The Batch CSV page offers a template. It accepts a file of at most **2 MB** and **2,000 rows**, validates each row and provides results CSV for download. Three example preset rows are available in the template.

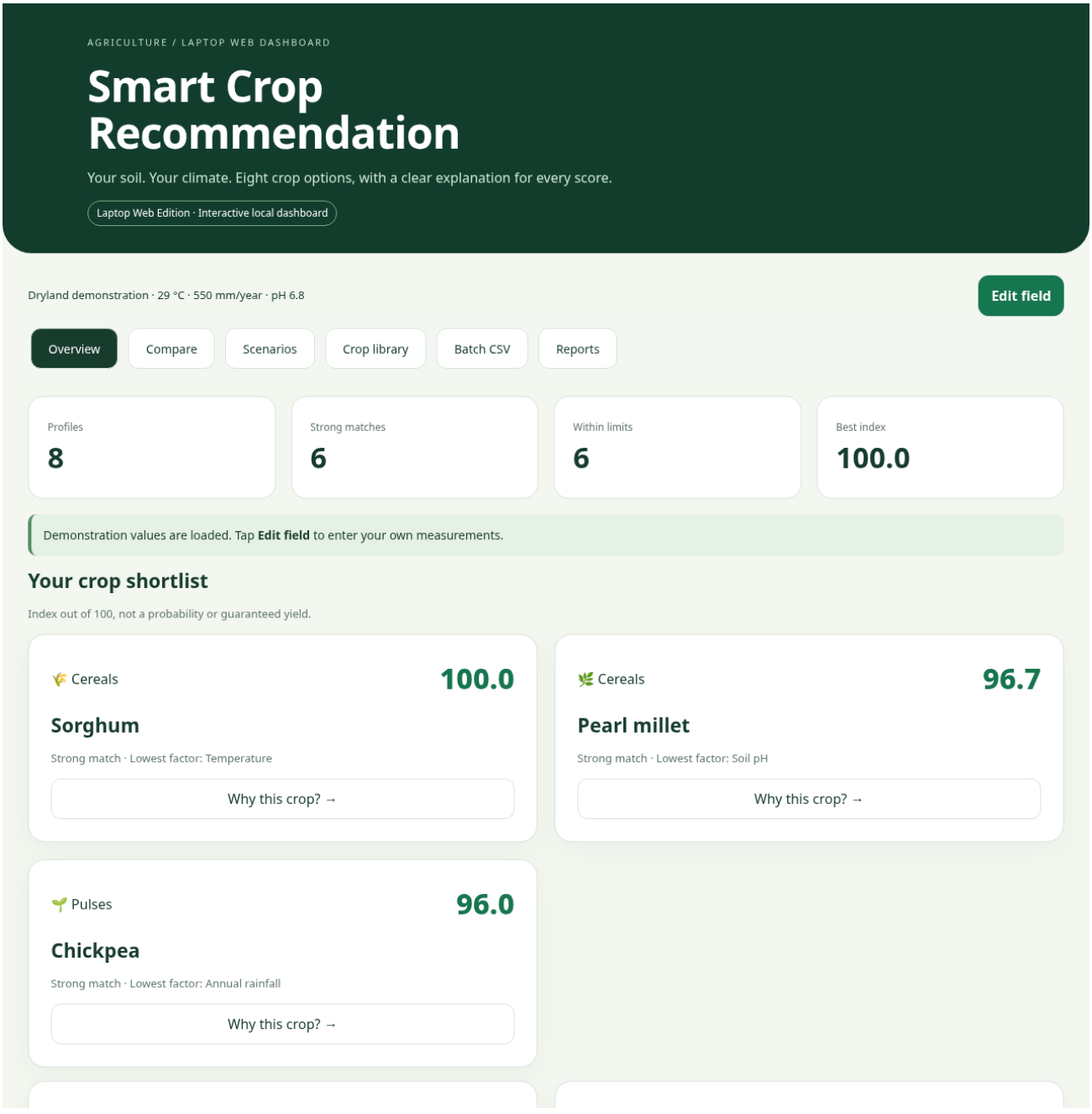
Suggested acceptance checks

Load each example preset; compare the top crop(s) with Chapter 9; set an invalid temperature and verify the form error; download a CSV; test one batch row; then download a printable report and confirm that it opens in the browser.

APPENDIX C / ACTUAL APPLICATION SCREENSHOTS

C.1 - Overview & interactive shortlist

Overview screen: four KPI cards, ranked crop shortlist, bar and donut summaries. Crop scores are the dryland demo output.



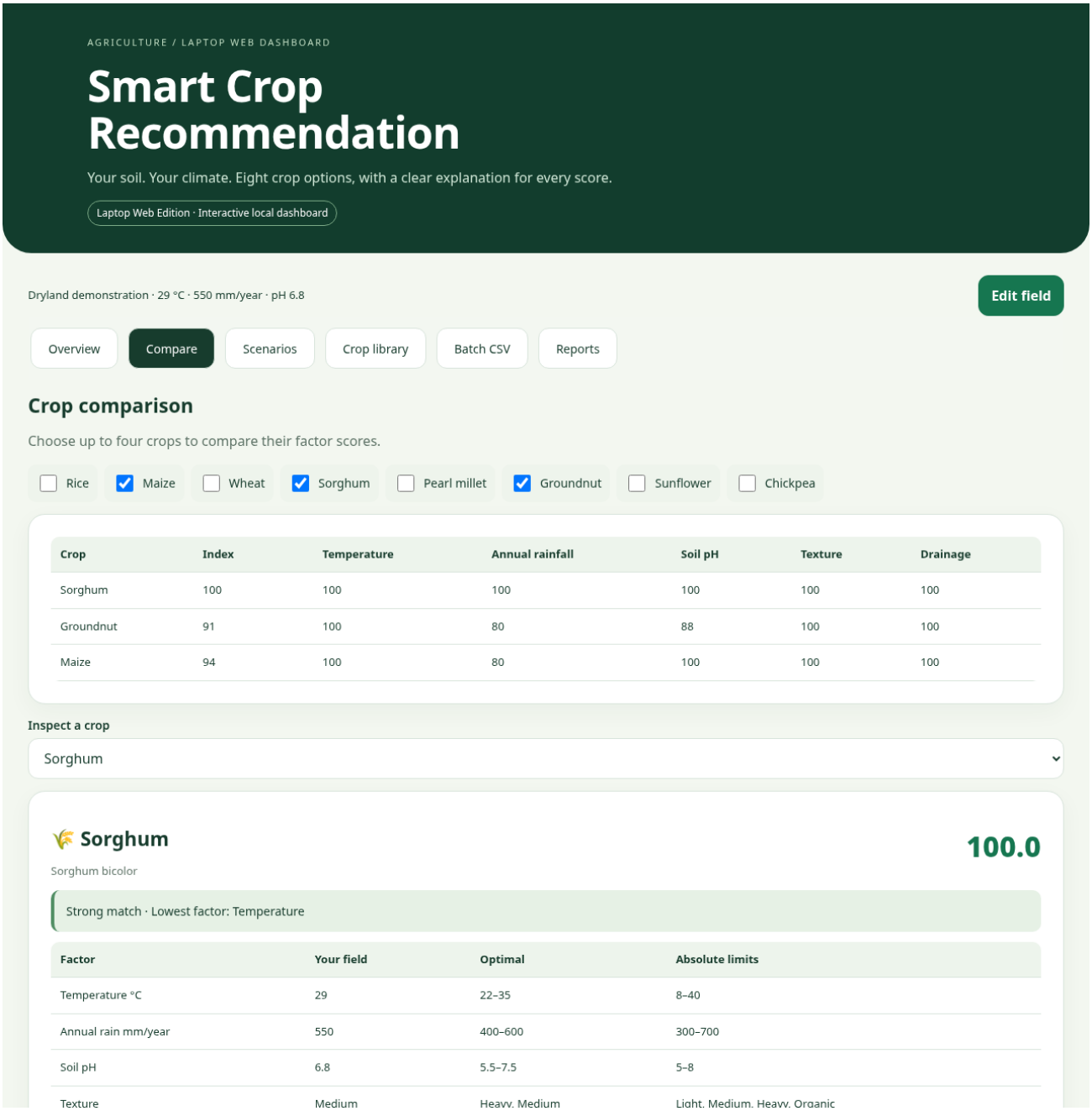
Screenshot C.1. Captured directly from the HTML interface embedded in Smart_Crop_Dashbord.py; desktop-browser view, 2026.

Note: the screenshots show design and implementation behavior, not verified farming outcomes.

APPENDIX C / ACTUAL APPLICATION SCREENSHOTS

C.2 - Crop comparison

Compare view: selected profiles, index tables and detailed factor explanations.



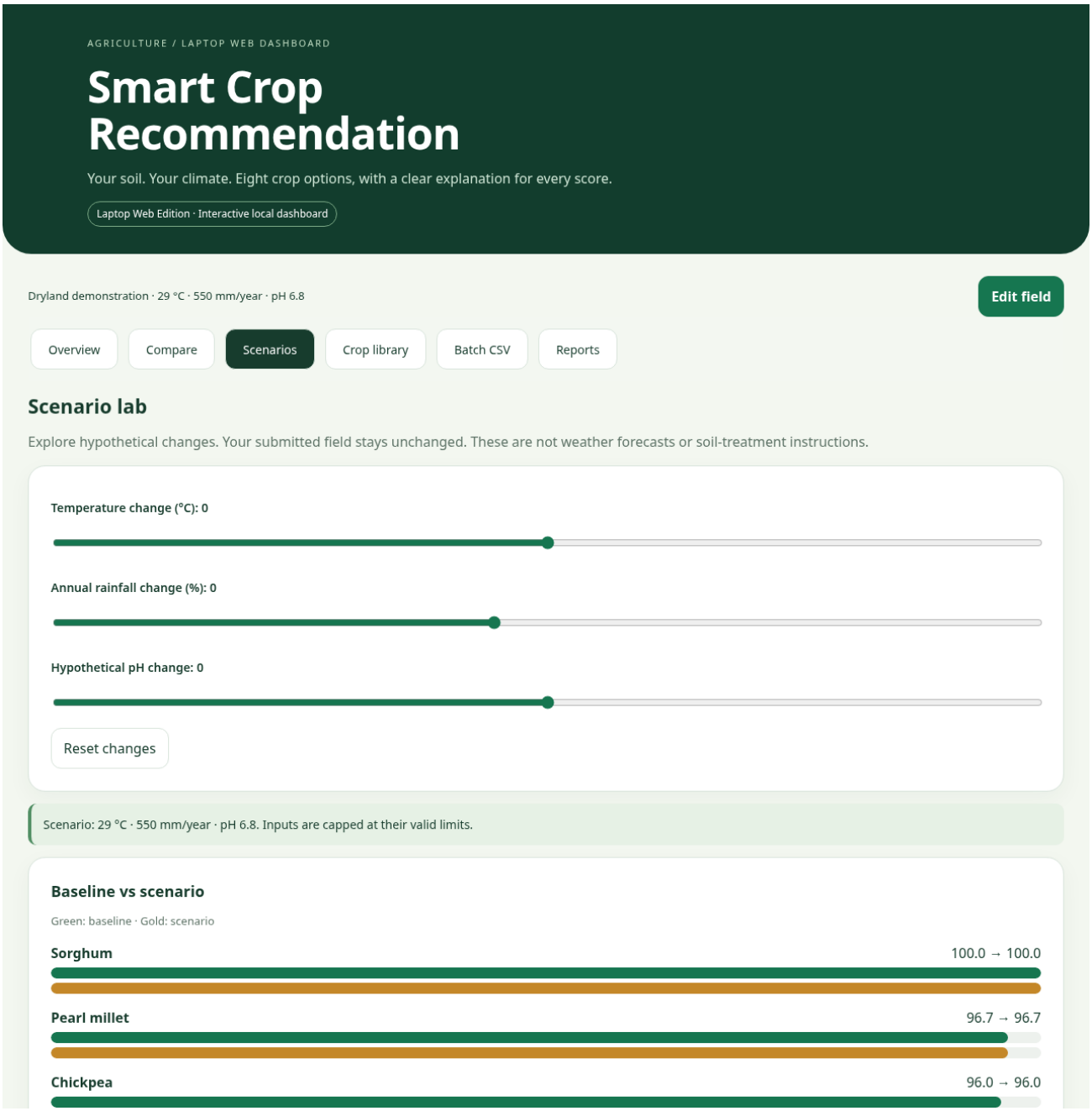
Screenshot C.2. Captured directly from the HTML interface embedded in Smart_Crop_Dashbord.py; desktop-browser view, 2026.

Note: the screenshots show design and implementation behavior, not verified farming outcomes.

APPENDIX C / ACTUAL APPLICATION SCREENSHOTS

C.3 - Scenario laboratory

Scenario lab: hypothetical field changes and temperature sensitivity tools.



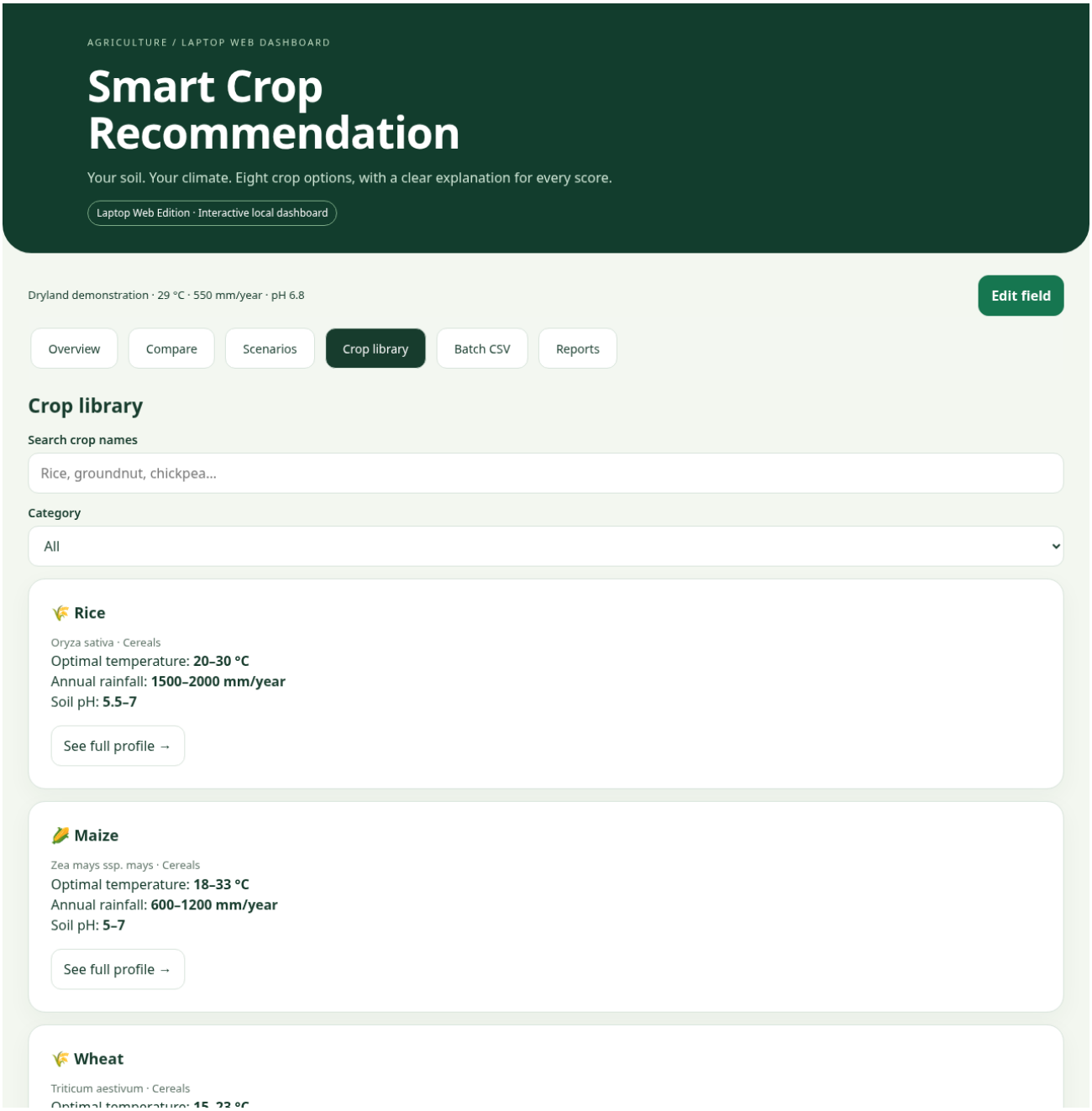
Screenshot C.3. Captured directly from the HTML interface embedded in Smart_Crop_Dashbord.py; desktop-browser view, 2026.

Note: the screenshots show design and implementation behavior, not verified farming outcomes.

APPENDIX C / ACTUAL APPLICATION SCREENSHOTS

C.4 - Searchable crop library

Crop library: eight reference profiles with scientific names, filters and field limits.



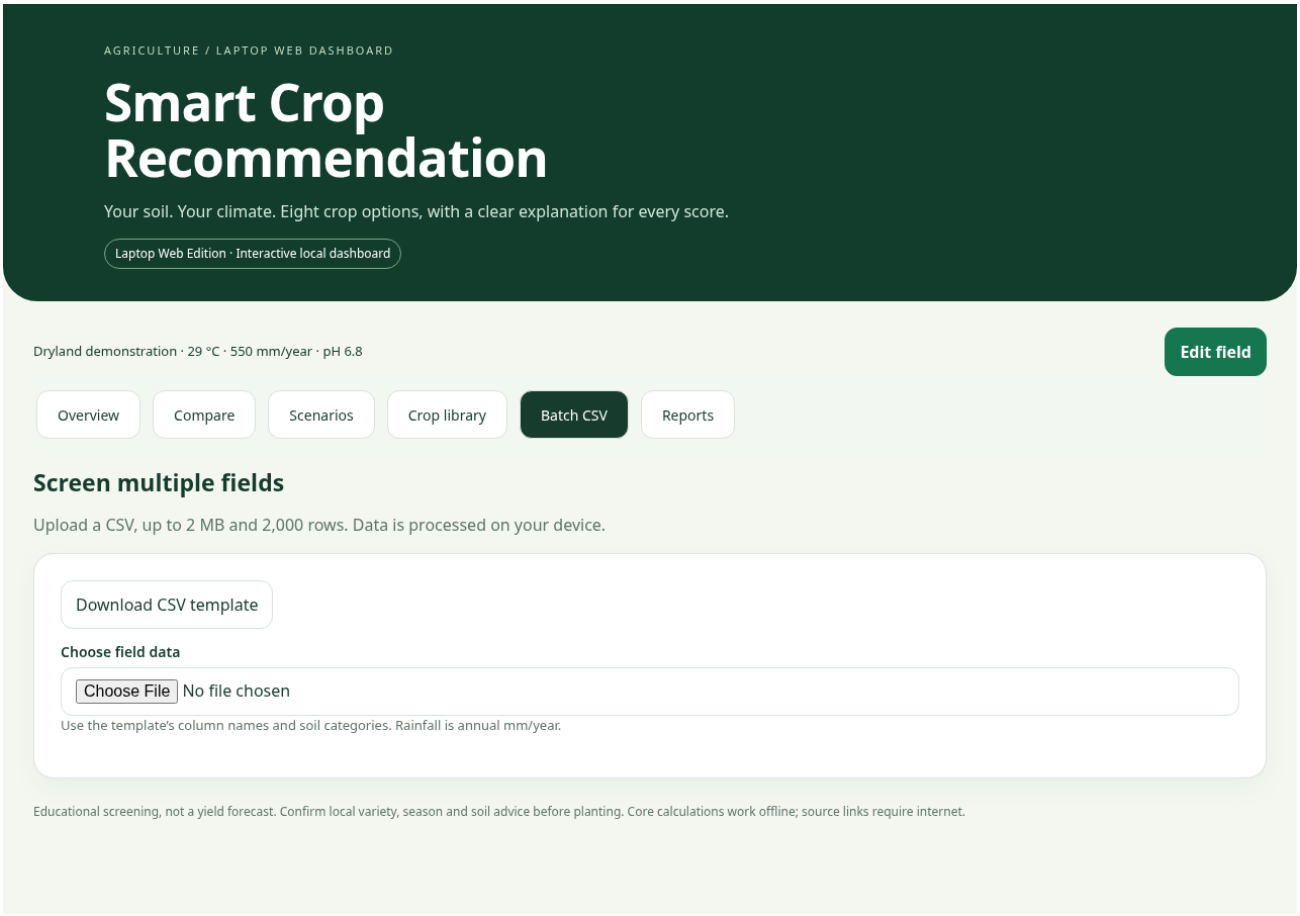
Screenshot C.4. Captured directly from the HTML interface embedded in Smart_Crop_Dashbord.py; desktop-browser view, 2026.

Note: the screenshots show design and implementation behavior, not verified farming outcomes.

APPENDIX C / ACTUAL APPLICATION SCREENSHOTS

C.5 - CSV batch screening

Batch CSV: downloadable template, field upload and device-local processing.



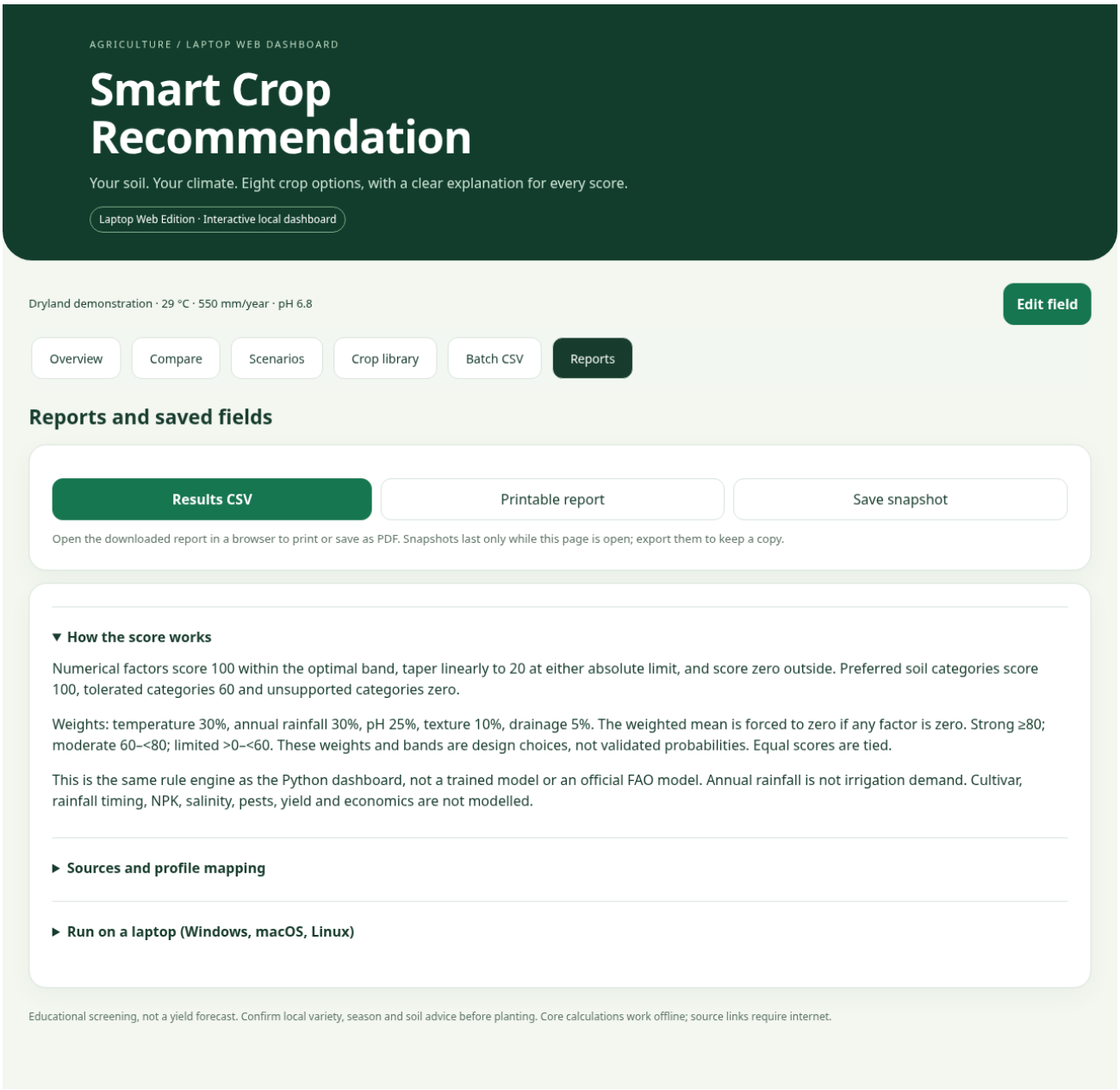
Screenshot C.5. Captured directly from the HTML interface embedded in Smart_Crop_Dashbord.py; desktop-browser view, 2026.

Note: the screenshots show design and implementation behavior, not verified farming outcomes.

APPENDIX C / ACTUAL APPLICATION SCREENSHOTS

C.6 - Reporting tools

Reports: results CSV, printable HTML report and temporary field snapshots.



Screenshot C.6. Captured directly from the HTML interface embedded in Smart_Crop_Dashbord.py; desktop-browser view, 2026.

Note: the screenshots show design and implementation behavior, not verified farming outcomes.